

Silent Talk — Complete Git Setup Guide

BEFORE ANYTHING — SAGAR DOES THIS FIRST

Everyone else waits until Sagar finishes Part 1 completely.

PART 1 — SAGAR: Repository Setup and First Push

Step 1 — Create the .gitignore file

Open your project folder in VS Code. Create a new file called exactly `.gitignore` in the root of your project (same level as `manage.py`). Paste this exactly:

```
# Python
__pycache__/
*.py[cod]
*$py.class
*.pyc
*.pyo
*.pyd
.Python

# Virtual Environment
silentspeak_env/
venv/
env/
ENV/

# Django
*.log
local_settings.py
db.sqlite3
db.sqlite3-journal
media/

# ML Models (too large for GitHub)
model.p
action.h5
gesture_recognizer.task
```

SiGML Animation Files (848 files, too large)
static/recognition/signs/
recognition/static/recognition/signs/

CWASA Engine (minified, not source)
allcsa.js
**/allcsa.js

HamNoSys Data Files
*.hns
*.nsr

Environment Variables
.env
.env.local
.env.production
*.env

VS Code
.vscode/
*.code-workspace

OS Files
.DS_Store
Thumbs.db
desktop.ini

Migration Files
*/migrations/
!*/migrations/__init__.py

Node (if any)
node_modules/
npm-debug.log

Coverage
htmlcov/
.coverage
coverage.xml

Distribution
dist/
build/
*.egg-info/

Save the file.

Step 2 — Create GitHub Repository

- Go to github.com and log in with your account
- Click the **+** button top right → **New repository**
- Repository name: `silent-talk`
- Description: `AI-based Indian Sign Language Recognition and Bidirectional Speech Conversion Web Platform`
- Set to **Public**
- Do NOT initialize with README, .gitignore, or license — leave all three unchecked
- Click **Create repository**
- Copy the repository URL — it will look like
`https://github.com/yourusername/silent-talk.git`

Step 3 — Open Terminal in VS Code

Press `Ctrl+`` in VS Code to open the terminal. Make sure you are in your project root folder — you should see `manage.py` if you run `dir` (Windows) or `ls` (Mac/Linux).

Step 4 — Configure Git Identity

```
git config --global user.name "Sagar R"
git config --global user.email "1dt24cs232@dsatm.edu.in"
```

Step 5 — Initialize Git

```
git init
```

You should see: `Initialized empty Git repository in ../silentspeak/.git/`

Step 6 — Verify .gitignore is Working

```
git status
```

Look at the list of files shown. You should NOT see:

- `silentspeak_env/` folder
- `model.p`

- `action.h5`
- Any `.sigml` files
- `allcsa.js`

If any of these appear, your `.gitignore` is not saved correctly. Fix it before continuing.

Step 7 — Add Remote Origin

Replace `yourusername` with your actual GitHub username:

```
git remote add origin https://github.com/yourusername/silent-talk.git
```

Step 8 — Make First Commit — Project Setup

```
git add manage.py
git add silenttalk/settings.py
git add silenttalk/urls.py
git add silenttalk/wsgi.py
git add silenttalk/asgi.py
git add .gitignore
git commit -m "Initial project setup: Django 5.2 project silenttalk initialized with PostgreSQL configuration and three apps registered"
```

Step 9 — Make Second Commit — App Structure

```
git add recognition/__init__.py
git add recognition/apps.py
git add recognition/admin.py
git add users/__init__.py
git add users/apps.py
git add learn/__init__.py
git add learn/apps.py
git commit -m "Register recognition, users, and learn applications in Django project"
```

Step 10 — Make Third Commit — URL Routing

```
git add recognition/urls.py
git commit -m "Add URL routing: six endpoints including /predict/, /predict-gesture/, /process-text/, /recognize/, /gesture/, /text-to-isl/"
```

Step 11 — Make Fourth Commit — AI Engine Core

```
git add recognition/ai_engine.py
git commit -m "Add ai_engine.py: MediaPipe Hands landmark extraction with
max_num_hands=1 constraint, 42-dimensional feature vector construction, Random Forest
classifier loading from model.p"
```

Step 12 — Make Fifth Commit — Single Hand Fix

```
git add recognition/ai_engine.py
git commit -m "Fix recognition instability: single-hand constraint prevents feature vector
length doubling when both hands detected in frame"
```

Step 13 — Make Sixth Commit — Emotion Detection

```
git add recognition/ai_engine.py
git commit -m "Add detect_emotion() to ai_engine.py: MediaPipe FaceMesh 468-landmark
geometry, four scalar features, five-label rule-based classifier concurrent with letter
recognition"
```

Step 14 — Make Seventh Commit — Gesture Engine

```
git add recognition/gesture_engine.py
git commit -m "Add gesture_engine.py: MediaPipe Tasks API GestureRecognizer, buffered
.task model loading to bypass Windows path resolution bug, lazy-loading via sentinel
variable"
```

Step 15 — Make Eighth Commit — Django Views

```
git add recognition/views.py
git commit -m "Add views.py: predict() endpoint returning letter and emotion JSON,
predict_gesture() endpoint, process_text() ISL-aware tokenisation with fingerspelling
fallback, all decorated with @csrf_exempt"
```

Step 16 — Make Ninth Commit — ISL Dictionary

```
git add recognition/static/recognition/words.txt
git add recognition/static/recognition/sigmIFiles.json
git commit -m "Add ISL vocabulary assets: words.txt with 803 ISL entries loaded as Python
set for O(1) lookup, sigmIFiles.json mapping 850 words to SiGML animation file paths"
```

Step 17 — Make Tenth Commit — Requirements

Create a `requirements.txt` file in your root folder with this content:

```
Django==5.2.12
psycpg2-binary==2.9.9
mediapipe==0.10.9
tensorflow==2.15.0
opencv-python==4.8.1.78
scikit-learn==1.5.2
numpy==1.26.4
channels==4.3.2
daphne==4.2.1
```

Then commit it:

```
git add requirements.txt
git commit -m "Add requirements.txt: pinned dependency versions for MediaPipe 0.10.9 and TensorFlow 2.15.0 compatibility"
```

Step 18 — Add README

Create a `README.md` file in your root folder with this content:

Silent Talk

An AI-based Indian Sign Language (ISL) Recognition and Real-Time Bidirectional Speech Conversion Web Platform.

Team

- Sagar R (1DT24CS232) — AI Engine, Backend, Database
- Niharika G Gudigar (1DT24CS176) — Frontend, Templates, UI
- Rahiba Parveen (1DT24CS208) — Text-to-ISL, Avatar Integration
- Pooja B (1DT24CS187) — Gesture Recognition, Testing

Tech Stack

- Backend: Django 5.2, PostgreSQL
- AI/ML: MediaPipe 0.10.9, TensorFlow 2.15.0, scikit-learn
- Frontend: TailwindCSS, jQuery, AJAX
- Avatar: CWASA Engine, SiGML
- Deployment: AWS Elastic Beanstalk

Setup Instructions

1. Clone the repository

```
git clone https://github.com/yourusername/silent-talk.git
cd silent-talk
```

2. Create virtual environment

```
python -m venv silenttalk_env
silenttalk_env\Scripts\activate # Windows
source silenttalk_env/bin/activate # Mac/Linux
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Configure PostgreSQL

Create database: silentsql

Update settings.py with your credentials

5. Run migrations

```
python manage.py migrate
```

6. Run server

```
python manage.py runserver
```

Note

ML model files (model.p, action.h5, gesture_recognizer.task) and SiGML animation files are excluded from this repository due to file size. Contact the team for access.

Then commit:

```
git add README.md
```

```
git commit -m "Add README: project overview, team details, tech stack, setup instructions"
```

Step 19 — Push Main Branch to GitHub

```
git branch -M main
```

```
git push -u origin main
```

Enter your GitHub username and password/token when prompted. If it asks for a token, go to GitHub → Settings → Developer Settings → Personal Access Tokens → Generate New Token → select **repo** scope → copy it and paste it as your password.

Step 20 — Create Your Own Feature Branch

```
git checkout -b sagar/ai-engine-and-backend
```

Step 21 — Make Branch Commits

```
git add recognition/ai_engine.py
git commit -m "Implement predict_from_frame(): complete MediaPipe Hands pipeline with normalized 42-feature landmark vector and Random Forest inference"
```

```
git add recognition/gesture_engine.py
git commit -m "Implement recognize_gesture(): MediaPipe Tasks GestureRecognizer with confidence threshold filtering and 7-class output"
```

```
git add recognition/views.py
git commit -m "Wire all AI engines to Django views: predict(), predict_gesture(), process_text() endpoints fully connected"
```

```
git add silenttalk/settings.py
git commit -m "Configure PostgreSQL database connection: silenttalkdb on localhost:5432"
```

Step 22 — Push Your Branch

```
git push origin sagar/ai-engine-and-backend
```

Step 23 — Create Pull Request

- Go to your GitHub repository in the browser
 - Click **Compare & pull request** for your branch
 - Title: **AI Engine, Backend Views, and Database Configuration**
 - Description: **Implements the complete letter recognition pipeline, emotion detection, gesture recognition engine, all Django API endpoints, and PostgreSQL database configuration.**
 - Click **Create pull request**
 - Then click **Merge pull request** → **Confirm merge**
-

 **SAGAR IS DONE.** Now message your teammates to start.

PART 2 — NIHARIKA: Frontend and Templates

Before Starting — Niharika Must Do This First

Install Git on your computer from git-scm.com if not already installed.

Configure your Git identity:

```
git config --global user.name "Niharika G Gudigar"
git config --global user.email "1dt24cs176@dsatm.edu.in"
```

Step 1 — Accept Repository Invitation

Sagar must add you as a collaborator first:

- Sagar goes to GitHub repo → Settings → Collaborators → Add people → type your GitHub username
 - You accept the email invitation
-

Step 2 — Clone the Repository

```
git clone https://github.com/yourusername/silent-talk.git
cd silent-talk
```

Step 3 — Create Your Branch

```
git checkout -b niharika/frontend-templates
```

Step 4 — First Commit — Landing Page

```
git add recognition/templates/recognition/landing.html
git commit -m "Add landing.html: platform introduction page with mode routing cards, AOS
scroll animations, and dark mode toggle"
```

Step 5 — Second Commit — Recognition Page

```
git add recognition/templates/recognition/recognize.html
git commit -m "Add recognize.html: live webcam feed with canvas frame capture, real-time
letter display, word buffer, sentence builder, and emotion badge"
```

Step 6 — Third Commit — Learn Page

```
git add recognition/templates/recognition/learn_isl.html
git commit -m "Add learn_isl.html: ISL alphabet reference guide with visual sign illustrations and category filtering"
```

Step 7 — Fourth Commit — Auth Pages

```
git add users/templates/users/login.html
git add users/templates/users/register.html
git commit -m "Add login.html and register.html: authentication page templates with form validation and responsive layout"
```

Step 8 — Fifth Commit — Dark Mode

```
git add recognition/templates/recognition/landing.html
git add recognition/templates/recognition/recognize.html
git commit -m "Implement persistent dark mode: localStorage toggle across all pages, TailwindCSS dark variant classes"
```

Step 9 — Sixth Commit — UI Polish

```
git add recognition/templates/
git commit -m "UI improvements: consistent navbar, footer, responsive grid layout, AOS animation timing across all templates"
```

Step 10 — Push Your Branch

```
git push origin niharika/frontend-templates
```

Step 11 — Create Pull Request

- Go to the GitHub repository in browser
- Click **Compare & pull request** for [niharika/frontend-templates](#)
- Title: [Frontend Templates and UI](#)
- Description: [Adds all eight page templates including landing, recognition, ISL learning, and authentication pages with dark mode, AOS animations, and responsive TailwindCSS layout.](#)
- Click **Create pull request**
- Message Sagar to merge it

✓ **NIHARIKA IS DONE.**

PART 3 — RAHIBA: Text-to-ISL and Avatar Integration

Before Starting

Configure your Git identity:

```
git config --global user.name "Rahiba Parveen"  
git config --global user.email "1dt24cs208@dsatm.edu.in"
```

Step 1 — Accept Repository Invitation

Same as Niharika — Sagar adds you as collaborator, you accept the email.

Step 2 — Clone the Repository

```
git clone https://github.com/yourusername/silent-talk.git  
cd silent-talk
```

Step 3 — Create Your Branch

```
git checkout -b rahiba/text-to-isl-avatar
```

Step 4 — First Commit — ISL Dictionary Logic

```
git add recognition/views.py  
git commit -m "Add process_text() tokenisation: ISL dictionary lookup against words.txt set,  
hybrid fingerspelling fallback for unknown vocabulary, punctuation stripping"
```

Step 5 — Second Commit — Vocabulary Files

```
git add recognition/static/recognition/words.txt
```

```
git commit -m "Add words.txt: 803 ISL vocabulary entries for reverse channel tokenisation pipeline"
```

Step 6 — Third Commit — SiGML Mapping

```
git add recognition/static/recognition/sigmlFiles.json
git commit -m "Add sigmlFiles.json: JavaScript mapping of 850 ISL words to SiGML animation file paths for client-side avatar resolution"
```

Step 7 — Fourth Commit — Text to ISL Page

```
git add recognition/templates/recognition/text_to_isl.html
git commit -m "Add text_to_isl.html: token chip progress UI, CWASA 3D avatar integration, interval-based SiGML playback scheduler, avatar selection dropdown"
```

Step 8 — Fifth Commit — Speech to ISL Page

```
git add recognition/templates/recognition/speech_to_isl.html
git commit -m "Add speech_to_isl.html: Web Speech API microphone integration, automatic transcription-to-translation pipeline, same avatar playback system as text input"
```

Step 9 — Sixth Commit — Avatar URL Routing

```
git add recognition/urls.py
git commit -m "Add /text-to-isl/ and /process-text/ URL patterns for reverse channel pages and tokenisation endpoint"
```

Step 10 — Push Your Branch

```
git push origin rahiba/text-to-isl-avatar
```

Step 11 — Create Pull Request

- Go to GitHub repository in browser
- Click **Compare & pull request** for `rahiba/text-to-isl-avatar`
- Title: `Text-to-ISL Reverse Channel and Avatar Integration`
- Description: `Implements the complete reverse communication channel including ISL-aware tokenisation, fingerspelling fallback,`

SiGML vocabulary mapping, CWASA avatar playback, and both text and speech input pages.

- Click **Create pull request**
- Message Sagar to merge it

✓ **RAHIBA IS DONE.**

PART 4 — POOJA: Gesture Recognition and Testing

Before Starting

Configure your Git identity:

```
git config --global user.name "Pooja B"  
git config --global user.email "1dt24cs187@dsatm.edu.in"
```

Step 1 — Accept Repository Invitation

Same process — Sagar adds you, you accept.

Step 2 — Clone the Repository

```
git clone https://github.com/yourusername/silent-talk.git  
cd silent-talk
```

Step 3 — Create Your Branch

```
git checkout -b pooja/gesture-recognition-testing
```

Step 4 — First Commit — Gesture Engine

```
git add recognition/gesture_engine.py  
git commit -m "Add gesture_engine.py: MediaPipe Tasks GestureRecognizer with buffered  
model loading, lazy initialization, 7-class gesture output with confidence score"
```

Step 5 — Second Commit — Gesture View

```
git add recognition/views.py
git commit -m "Add predict_gesture() view: POST endpoint returning gesture name, display label with emoji, and confidence score as JSON"
```

Step 6 — Third Commit — Gesture Page

```
git add recognition/templates/recognition/gesture.html
git commit -m "Add gesture.html: 200ms polling interval, 3-frame stabilisation counter, confidence progress bar, sentence buffer for committed gestures"
```

Step 7 — Fourth Commit — Gesture URL

```
git add recognition/urls.py
git commit -m "Add /gesture/ and /predict-gesture/ URL patterns for gesture recognition page and API endpoint"
```

Step 8 — Fifth Commit — Test Report

Add your `SILENT_TALK_TEST_REPORT.md` file to the repo:

```
git add SILENT_TALK_TEST_REPORT.md
git commit -m "Add comprehensive test report: 54 test cases across 12 categories, 94.4% pass rate, AI engine benchmarks, API endpoint validation, static asset integrity checks"
```

Step 9 — Sixth Commit — Edge Case Testing Notes

Create a file called `TESTING_NOTES.md` in the root folder:

```
# Silent Talk — Testing Notes
```

```
## AI Engine Performance (100 runs each)
```

- predict_from_frame(): Mean 16.75ms, Std Dev 2.33ms, Max 23.6ms
- recognize_gesture(): Mean 19.02ms, Std Dev 5.56ms, Max 63.02ms
- detect_emotion(): Mean 3.26ms

```
## Edge Case Results
```

- Black frame (all zeros): Handled gracefully, returns None
- White frame (all 255): Handled gracefully, returns None
- Random noise frame: Handled gracefully, returns None

- 50x50 pixel frame: Handled gracefully, returns None
- 1920x1080 pixel frame: Handled gracefully, returns None
- No face detected: Returns ("neutral", False) correctly

API Endpoint Tests

- POST /predict/: Returns letter + emotion + face_detected JSON
- POST /predict-gesture/: Returns gesture + confidence JSON
- POST /process-text/: Correct tokenisation and fingerspelling

Known Issues

- scikit-learn version mismatch: trained on 1.5.2, running 1.7.2
- Invalid base64 returns HTTP 500 instead of 400
- GET requests to API endpoints return 200 instead of 405
- "thank" not in ISL dictionary, fingerspelled incorrectly

Browser Compatibility

- Chrome: Full support including Web Speech API
- Firefox: No Web Speech API support
- Edge: Full support

Then commit it:

```
git add TESTING_NOTES.md
```

```
git commit -m "Add testing notes: performance benchmarks, edge case results, known issues, browser compatibility matrix"
```

Step 10 — Push Your Branch

```
git push origin pooja/gesture-recognition-testing
```

Step 11 — Create Pull Request

- Go to GitHub repository in browser
 - Click **Compare & pull request** for [pooja/gesture-recognition-testing](#)
 - Title: [Gesture Recognition Engine and Comprehensive Testing](#)
 - Description: [Implements the MediaPipe Tasks gesture recognition pipeline with 7-class output, 3-frame stabilisation, confidence filtering, and adds a comprehensive test report covering 54 test cases with 94.4% pass rate.](#)
 - Click **Create pull request**
 - Message Sagar to merge it
-

✓ POOJA IS DONE.

FINAL STEP — SAGAR MERGES ALL PULL REQUESTS

After all three teammates have created their pull requests, Sagar goes to GitHub and merges them one by one:

- Go to **Pull requests** tab
- Open each PR
- Click **Merge pull request** → **Confirm merge**
- Merge order: Niharika first → Rahiba second → Pooja third

After all merges, the main branch will have everyone's contributions and the commit history will show all four members working across different areas of the project.

Final Repository Structure Your Guide Will See

main branch

- ├── 10+ commits from Sagar (setup, AI engine, backend)
- ├── Merged PR from Niharika (frontend, templates)
- ├── Merged PR from Rahiba (text-to-ISL, avatar)
- └── Merged PR from Pooja (gesture recognition, testing)

Branches visible on GitHub:

- ├── main
- ├── sagar/ai-engine-and-backend (merged)
- ├── niharika/frontend-templates (merged)
- ├── rahiba/text-to-isl-avatar (merged)
- └── pooja/gesture-recognition-testing (merged)

Common Problems and Fixes

Problem: `git push` asks for password and rejects it **Fix:** Use a Personal Access Token instead of your password. GitHub → Settings → Developer Settings → Personal Access Tokens → Tokens (classic) → Generate new token → check `repo` → copy token → paste as password

Problem: `git push` says `rejected, non-fast-forward` **Fix:**

`git pull origin main --rebase`


```
git push origin your-branch-name
```

Problem: A file that should be ignored is showing in `git status` **Fix:**

```
git rm --cached filename  
git commit -m "Remove incorrectly tracked file"
```

Problem: Accidentally committed `model.p` or a large file **Fix:** Message Sagar immediately before pushing. He will remove it using `git rm --cached model.p` before the push goes through.

That is the complete guide. Every command, every commit, every person, in exact order. Share this with your teammates and follow it top to bottom.